# SJSU Zoo - Class Diagram

CS 151 Fall 2026 · all 43 classes in src/main · from thasin2/CS151-Fall2026-SJSUZoo (main @ d56819a, Oct 22, 2026) · private helpers, one-line getters/setters and equals / hashCode / toString not shown

«use»
«use»
«use»
«use»
«use»
«use»
«use»
«use»
«use»
«use»
«use»
«use»
«use»
«use»
«use»
«use»
0..1
1
0..*
0..1
0..*
0..1
0..*
0..*
1
0..*
0..*
0..*
1
1
1
1
3
0..*
0..*
1
1
0..*
1

## zoo.ui

**ZooBuilderMenu**
- game: ZooBuilderGame

+ run(): void

**ScheduleGameMenu**
- win: boolean

+ run(): void

**HabitatGameMenu**

+ run(): void

**AnimalGameMenu**

+ run(): void

**Main**

+ main(args: String[]): void

**ConsoleInput**

+ readLine(prompt: String): String
+ readNonEmptyLine(prompt: String): String
+ readInt(prompt: String, min: int, max: int): int
+ readYesNo(prompt: String): boolean

**ExitRequestedException**

**ManagerGameMenu**

+ run(): void

## zoo.game.zoobuilder

**ZooBuilderGame**
- random: Random
- zoo: Zoo

+ ZooBuilderGame(zooName: String, visitorCapacity: int)
+ addHabitat(name: String, capacity: int): void
+ checkCanAddAnimal(habitat: Habitat, name: String): void
+ addLandAnimal(habitat: Habitat, name: String, skinType: String): void
+ addWaterAnimal(habitat: Habitat, name: String, depth: int): void
+ addFlyingAnimal(habitat: Habitat, name: String, maxHeight: int): void
+ addAttraction(name: String, capacity: int): void
+ registerVisitor(name: String, age: int, money: int): Visitor
+ removeHabitat(habitat: Habitat): void
+ removeVisitor(name: String): Visitor
+ isReadyToOpen(): boolean
+ grandOpening(): void
+ findHabitat(name: String): Habitat
+ findAnimal(name: String): Animal
+ countAnimals(): int
+ getHabitats(): List<Habitat>

## zoo.game.schedule

**ScheduleGame**
- inventory: Inventory
- attraction: Attraction
- random: Random
- visitors: ArrayList<Visitor>
- leftHungry: int
- health: boolean

+ ScheduleGame()
+ setUpAttraction(name: String, capacity: int): void
+ displayAttraction(): void
+ dayCycle(day: int): void
+ maintenanceWork(): void
+ failCheck(): boolean
+ finalFailCheck(): boolean

## zoo.game.habitat

**HabitatGame**
- habGameInv: Inventory
- zooHabGame: Zookeeper
- maintHabGame: MaintenanceStaff
- temp: AnimalGame
- opened: boolean

+ makeHabitat(): Habitat
+ firstPart(hab: Habitat): void
+ secondPart(hab: Habitat): boolean
+ option2(hab: Habitat): boolean
+ option3(hab: Habitat): boolean
+ add(x: Habitat): void
+ open(x: Habitat): void
+ close(hab: Habitat): void
+ maintenance(hab: Habitat, stor: Inventory): void
+ stockingProcesses(item: String): void
+ feed(a: Animal): void

## zoo.game.animal

**AnimalGame**
- aniGameInv: Inventory
- vetAniGame: Veterinarian
- zooAniGame: Zookeeper

+ nameAnimal(): String
+ createAnimalObj(name: String): Animal
+ firstPart(a: Animal): void
+ healingAnimalProcess(a: Animal): void
+ stockingProcess(item: String): void
+ secondPart(a: Animal): boolean
+ feed(a: Animal): void

## zoo.game.manager

«enumeration»
**ManagerLevel**
LEVEL_1
LEVEL_2
LEVEL_3
- number: int
- weeksToWin: int
- startingBudget: double
- startingIncome: double
- incomeGrowth: double
- bonusAmount: double
- eventChance: double
- maxEventsPerWeek: int
- scoreMultiplier: int

+ fromNumber(number: int): ManagerLevel
+ getIncomeForWeek(week: int): double

«enumeration»
**ManagerEvent**
ANIMAL_EMERGENCY
DANGEROUS_ANIMAL_WORK

**ManagerGame**
- level: ManagerLevel
- random: Random
- manager: Manager
- animals: ArrayList<Animal>
- zookeeperDemand: ManagerRoleDemand
- vetDemand: ManagerRoleDemand
- maintenanceDemand: ManagerRoleDemand
- eventMessages: ArrayList<String>
- budget: double
- week: int
- emergencies: int
- emergencyAnimal: Animal
- hazardShifts: int
- dangerousAnimal: Animal
- bonusWeek: boolean
- bonusesEarned: int
- weeksSurvived: int
- over: boolean
- won: boolean
- nextZookeeperNumber: int
- nextVetNumber: int
- nextMaintenanceNumber: int

+ ManagerGame(level: ManagerLevel, managerName: String, random: Random)
+ hire(role: StaffRole, fullTime: boolean): Staff
+ fire(staffId: String): Staff
+ runWeek(): ManagerWeekReport
+ planShifts(role: StaffRole): ManagerShiftPlan
+ isForecastExactlyCovered(): boolean
+ getShiftCapacity(role: StaffRole, withOvertime: boolean): int
+ getMinimumShifts(role: StaffRole): int
+ getDemand(role: StaffRole): ManagerRoleDemand
+ getScore(): int
+ getWeekPoints(): int
+ getMoneyPoints(): int
+ getPointsPerWeek(): int
+ getMarketRate(role: StaffRole, fullTime: boolean): double
+ getRolesWithDemand(): StaffRole[]
+ getWeeksToWin(): int
+ getIncomeThisWeek(): double

**ManagerRoleDemand**
- start: int
- risePeriod: int
- base: int
- extra: int

+ ManagerRoleDemand(start: int, risePeriod: int)
+ startWeek(week: int): void
+ addExtra(shifts: int): void
+ getTotal(): int

**ManagerShiftPlan**
- workers: ArrayList<Staff>
- plannedShifts: int[]
- plannedHazardShifts: int[]
- demand: int
- emergencies: int
- regularCapacity: int
- filled: int
- hazardShiftsLeft: int
- carriedOut: boolean

+ ManagerShiftPlan(team: ArrayList<Staff>, demand: int, hazardShifts: int, emergencies: int)
+ getUnfilledShifts(): int
+ getOvertimeShifts(): int
+ getUnneededShifts(): int
+ isExactlyCovered(): boolean
carryOut(dangerousAnimal: Animal, emergencyAnimal: Animal): void

**ManagerWeekReport**
- week: int
- payStubs: ArrayList<PayStub>
- unfilledZookeeperShifts: int
- unfilledVetShifts: int
- unfilledMaintenanceShifts: int
- penalty: double
- income: double
- bonusWeek: boolean
- bonus: double
- budgetAfter: double

+ ManagerWeekReport(week: int, payStubs: ArrayList<PayStub>, unfilledZookeeperShifts: int, unfilledVetShifts: int, unfilledMaintenanceShifts: int, penalty: double, income: double, bonusWeek: boolean, bonus: double, budgetAfter: double)
+ getPayroll(): double
+ getUnfilledShifts(role: StaffRole): int

## java.lang

**RuntimeException**

**IllegalStateException**

**IllegalArgumentException**

## zoo.exception

**BudgetExceededException**

**InsufficientFundsException**

**JobLimitException**

**ShiftLimitException**

**ZooFullException**

**DuplicateEntryException**

**InvalidPayException**

**Thrown by**

| | |
|---|---|
| **BudgetExceededException** | ManagerGame |
| **InsufficientFundsException** | Visitor |
| **JobLimitException** | MaintenanceStaff |
| **ShiftLimitException** | Staff |
| **ZooFullException** | Manager, Zoo |
| **DuplicateEntryException** | Zoo |
| **InvalidPayException** | Staff |
| **ExitRequestedException** | ConsoleInput |

## zoo

**Zoo**
+ MAXIMUM_INSTANCES: int
- name: String
- capacity: int
- attractions: List<Attraction>
- registeredVisitors: List<Visitor>
- visitorsInside: List<Visitor>
- registeredAnimals: List<Animal>
- registeredHabitats: List<Habitat>
- open: boolean
- dayNumber: int
- visitorsToday: int
- ticketsSoldToday: int
- ticketRevenueToday: double
- totalTicketRevenue: double
- attractionVisitsAtStartOfDay: int

+ checkLimit(items: Collection<?>, itemType: String): void
+ Zoo(name: String, capacity: int)
+ addAttraction(attraction: Attraction): void
+ getAttraction(attractionName: String): Attraction
+ registerVisitor(visitor: Visitor): void
+ unregisterVisitor(visitor: Visitor): void
+ findVisitor(visitorName: String): Visitor
+ registerAnimal(animal: Animal): void
+ registerHabitat(habitat: Habitat): void
+ unregisterHabitat(habitat: Habitat): void
+ openForDay(): void
+ closeForTheDay(): void
recordTicketSale(visitor: Visitor): void
admitVisitor(visitor: Visitor): void
removeVisitor(visitor: Visitor): void
+ admitGroup(group: List<Visitor>): int
+ sendVisitorsToAttractions(chance: double, random: Random): int
+ printDailyReport(): void
+ getTotalAttractionVisits(): int
+ getAttractionVisitsToday(): int
+ isFull(): boolean
+ getVisitorCount(): int

**Visitor**
+ MAX_PATIENCE: int
- name: String
- age: int
- ticketType: TicketType
- wallet: double
- hasTicket: boolean
- insideZoo: boolean
- currentAttraction: Attraction
- patience: int

+ Visitor(name: String, age: int)
+ Visitor(name: String, age: int, startingMoney: double)
+ addFunds(amount: double): void
+ canAffordTicket(): boolean
+ buyTicket(zoo: Zoo): void
+ enterZoo(zoo: Zoo): void
+ leaveZoo(zoo: Zoo): void
+ visitAttraction(attraction: Attraction): boolean
+ leaveAttraction(): void
+ isUpset(): boolean
+ isAtAttraction(): boolean

«enumeration»
**TicketType**
CHILD
ADULT
SENIOR
- price: double

+ fromAge(age: int): TicketType

«interface»
**Maintainable**

+ *getCondition(): int*
+ *needsMaintenance(): boolean*
+ *performMaintenance(): void*

## zoo.attractions

**Attraction**
- isOpen: boolean
- totalVisited: int
- currentVisitors: ArrayList<Visitor>
- capacity: int
- condition: int
- name: String

+ Attraction()
+ Attraction(capacity: int)
+ admit(visitor: Visitor): boolean
+ open(): void
+ close(): void
+ getCurrentVisitors(): String
+ getCurrentVisitorCount(): int
+ getCurrentVisitor(): Visitor
+ remove(visitor: Visitor): boolean
+ isFull(): boolean
+ setName(name: String): void
+ needsMaintenance(): boolean
+ performMaintenance(): void

## zoo.habitat

**Habitat**
- name: String
- capacity: int
- animals: ArrayList<Animal>
- condition: int

+ Habitat(name: String, capacity: int, animals: ArrayList<Animal>)
+ Habitat(name: String, capacity: int, animal: Animal)
+ habitatOpen(): boolean
+ habitatClose(): void
+ ensureSafeAdd(animal: Animal): boolean
+ setCondition(condition: int): void
+ needsMaintenance(): boolean
+ performMaintenance(): void
+ addToAnimals(animal: Animal): boolean

## zoo.animal

**Animal**
- dangerous: boolean
- isHungry: boolean
- isActive: boolean
- name: String
- injured: boolean
- trickCount: int

+ Animal(name: String, dangerous: boolean)
+ Animal(name: String)
+ doATrick(): boolean
+ tired(trickCount: int): boolean
+ existInZoo(): boolean
+ setHungry(isHungry: boolean): boolean
+ setTrickCount(trickCount: int): void

**AquaticAnimals**
- depthResided: int

+ AquaticAnimals(name: String, dangerous: boolean, depthResided: int)
+ AquaticAnimals(name: String, depthResided: int)
+ doATrick(): boolean
+ existInZoo(): boolean

**TerrestrialAnimals**
- skinType: String

+ TerrestrialAnimals(name: String, dangerous: boolean, skinType: String)
+ TerrestrialAnimals(name: String, skinType: String)
+ doATrick(): boolean
+ existInZoo(): boolean

**SkyAnimal**
- maxHeight: int

+ SkyAnimal(name: String, dangerous: boolean, maxHeight: int)
+ SkyAnimal(name: String, maxHeight: int)
+ doATrick(): boolean
+ existInZoo(): boolean
+ setMaxHeight(maxHeight: int): void

## zoo.inventory

**Inventory**
- food: int
- medicine: int
- supplies: int
- foodPrice: double
- medicinePrice: double
- suppliesPrice: double
- budget: double

+ Inventory()
+ stock(item: String, amount: int): void
+ addBudget(amount: double): void
+ use(item: String): boolean

## zoo.staff

***Staff***
- id: String
- name: String
- pay: double
- fullTime: boolean
- shiftsThisWeek: int

# Staff(id: String, name: String, pay: double, fullTime: boolean)
*# calculateWeeklyPay(): double*
*+ getRole(): StaffRole*
# resetWeeklyCounts(): void
# getHoursWorked(): int
# addOvertimePay(straightTimePay: double): double
+ workShift(): void
+ closeOutWeek(): double
+ getMinimumShifts(): int
+ getRegularShiftLimit(): int
+ getMaximumShifts(): int

«enumeration»
**StaffRole**
ZOOKEEPER
VETERINARIAN
MAINTENANCE
MANAGER
- displayName: String

**PayStub**
- staffId: String
- staffName: String
- role: StaffRole
- shiftsWorked: int
- amount: double

+ PayStub(staffId: String, staffName: String, role: StaffRole, shiftsWorked: int, amount: double)

**Zookeeper**
- hazardShiftsThisWeek: int

+ Zookeeper(id: String, name: String, pay: double, fullTime: boolean)
# calculateWeeklyPay(): double
# resetWeeklyCounts(): void
+ workShiftWithAnimal(animal: Animal): void
+ feedAnimal(anim: Animal, storage: Inventory): boolean
+ getRole(): StaffRole

**Veterinarian**
- emergenciesThisWeek: int

+ Veterinarian(id: String, name: String, pay: double, fullTime: boolean)
# calculateWeeklyPay(): double
# resetWeeklyCounts(): void
+ handleEmergency(animal: Animal): void
+ getRole(): StaffRole
+ fixAnimal(animal: Animal, storage: Inventory): boolean

**MaintenanceStaff**
- jobsThisWeek: int

+ MaintenanceStaff(id: String, name: String, pay: double, fullTime: boolean)
# calculateWeeklyPay(): double
# resetWeeklyCounts(): void
+ service(target: Maintainable): void
+ hasJobsLeft(): boolean
+ getRole(): StaffRole

**Manager**
- team: ArrayList<Staff>

+ Manager(id: String, name: String, pay: double, fullTime: boolean)
# calculateWeeklyPay(): double
+ addTeamMember(member: Staff): void
+ removeTeamMember(member: Staff): void
+ findTeamMember(id: String): Staff
+ getTeamByRole(role: StaffRole): ArrayList<Staff>
+ runPayroll(): ArrayList<PayStub>
+ getRole(): StaffRole
+ getTeamSize(): int